



RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA
RECHERCHE SCIENTIFIQUE
ÉCOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

Project Advanced Database Systems & Big Data Mining

2^{ème} Année Cycle Supérieur (2CS)
Option : Systèmes Informatiques et Logiciels (SL)

Medical Time-Series Dataset Curation

Explainable Anomaly Detection: From Raw Signals to Validated Benchmarks

Réalisé par :

ATTIA Oussama Abderraouf
SRAICH Imene
BENCHIHEUB Zineb Yasmine
BOUZEGUELLA Imane
DJEBROUNI Wafa Feriel
MAHMOUDI Sameh
KETTOUCHE Yasmine
ROUABHI Mounia Hibet Errahmane
BENSEMANE Mohamed Reda
GUEROUMI Bahaa Eddine

Encadré par :

BENKRID Soumia
HAMDA Leila

Année universitaire : 2025–2026

0.1. Introduction

The UCI Human Activity Recognition (HAR) dataset presents a dual data structure representative of real-world physiological signal pipelines: on one side, raw inertial sensor readings sampled at **50 Hz** across 9 channels, and on the other, **561 pre-computed statistical features** aggregated over 2.56-second sliding windows. Managing both data types efficiently — at storage, query, and model-training time — requires two complementary database systems operating in parallel.

This document justifies the selection of **InfluxDB** as the secondary time-series database alongside PostgreSQL, and explains how the schema designed for each system addresses the specific challenges of time-series data at scale.

0.2. Choice of Secondary Database: InfluxDB

0.2.1 Why a Dedicated Time-Series Database?

PostgreSQL is a mature, general-purpose RDBMS that excels at structured data and complex queries. However, it was not designed for continuous sensor streams, which exhibit specific access patterns:

- **High write throughput:** $50 \text{ Hz} \times 9 \text{ channels} \times 30 \text{ subjects}$ produces sustained concurrent ingestion that stresses row-based B-Tree storage.
- **Time-range scans:** Retrieving all signal values between T_1 and T_2 is poorly served by a generic B-Tree index on a timestamp column.
- **Downsampling & compression:** Computing per-second averages and applying delta/RLE encoding require native TSM engine support that PostgreSQL lacks.

0.2.2 Why InfluxDB Specifically?

Criterion	InfluxDB	Alternatives
Data Model	Native TAG / FIELD / TIMESTAMP — no schema migration	TimescaleDB extends PG (more overhead); OpenTSDB needs HBase
Write Speed	Millions of points/sec via line-protocol batching	QuestDB is faster but has a less mature ecosystem
Query Language	Flux: pipeline-based with built-in <code>aggregateWindow()</code>	ClickHouse uses SQL but targets OLAP, not sensor streams
Python Client	Official <code>influxdb-client</code> with DataFrame write API	All alternatives have Python clients; InfluxDB's is best documented
Open Source	OSS 2.x fully free for research	All considered options are open source

0.3. Schema Design and Time-Series Challenges

0.3.1 Two-Measurement Design

The HAR dataset operates at two fundamentally different time scales. Forcing both into a single measurement would create sparsity. Two separate measurements solve this:

Measurement	Resolution	Content	Volume
har_kinematic_signals	20 ms (50 Hz)	9 raw signal fields	≈1.32 M points
har_windowed_features	2.56 s/window	561 engineered features	10,299 points

0.3.2 TSM Engine and TAG Indexing

InfluxDB stores data in a **Time-Structured Merge Tree (TSM)**: every write is appended in timestamp order, enabling range scans via sequential reads with no random I/O. Index shards are organised by time bucket so queries touching a narrow range open only relevant shards.

Every point is tagged with `subject_id`, `activity`, and `data_split`. InfluxDB builds an **inverted index** over tag values at write time, resolving a filter like `subject_id = "1" AND activity = "WALKING"` in **O(1) time** — compared to the multi-table JOIN + B-Tree scan required in PostgreSQL.

0.3.3 Native Downsampling and Self-Contained Points

Benchmark query Q4 requires resampling the 50 Hz signal to 1-second bins. In Flux this is a single native operator:

```
|> aggregateWindow(every: 1s, fn: mean)
```

The PostgreSQL equivalent requires `DATE_TRUNC + GROUP BY` after unpacking a `double precision[]` array — a significantly more expensive operation. Furthermore, every InfluxDB point encodes *who*, *what*, *when*, and *which split* directly in its tags, eliminating all join overhead during the model training data-fetch loop.

0.4. PostgreSQL vs. InfluxDB — Summary

Dimension	PostgreSQL	InfluxDB
Primary strength	Complex multi-table queries, integrity constraints	High-frequency append writes, time-range scans
Indexing	B-Tree on PKs and FKs	TSM time order + inverted tag index
Join support	Full SQL JOIN across 6 tables	No joins — context embedded in tags
Downsampling	DATE_TRUNC + GROUP BY (array unpack)	Native aggregateWindow() operator
Expected edge	Activity/feature metadata aggregations	Range scans, write throughput, downsampling

The quantitative verdict will be produced in the Sprint 2 Benchmark Report.